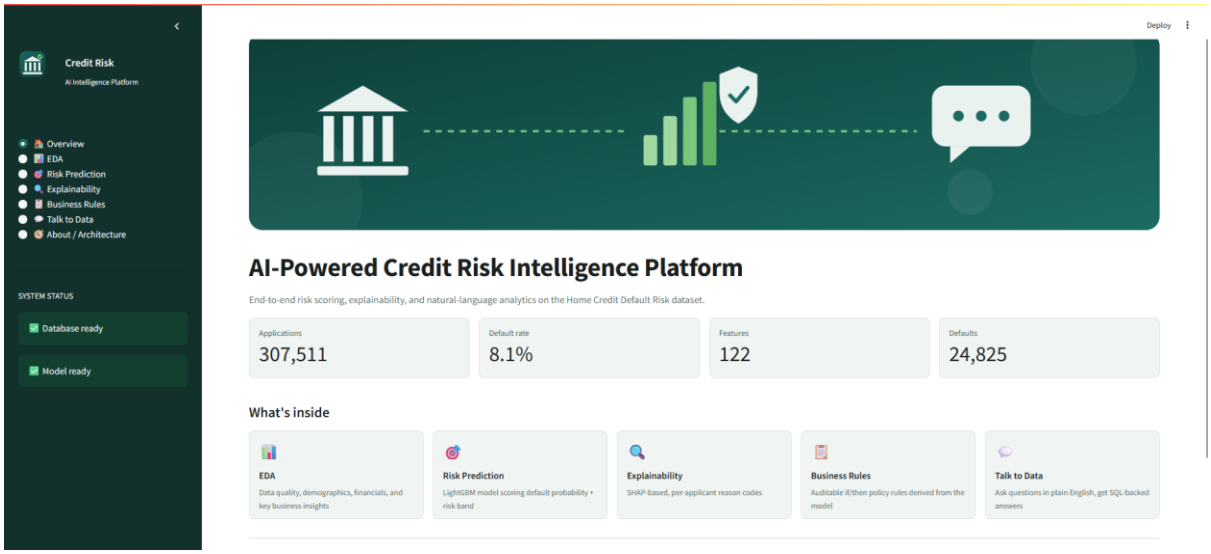
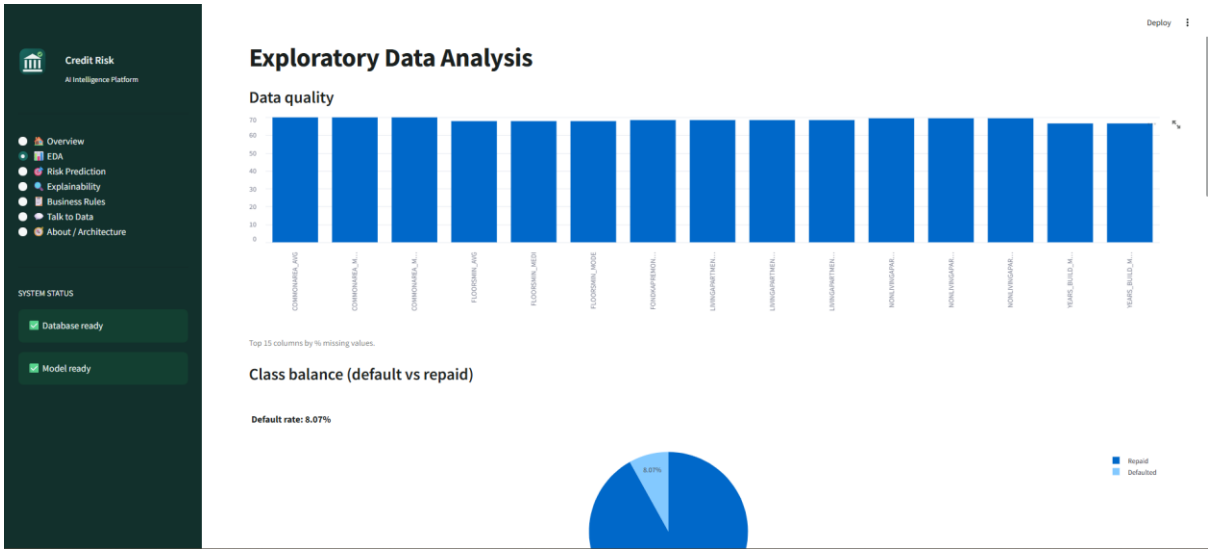
Project Presentation





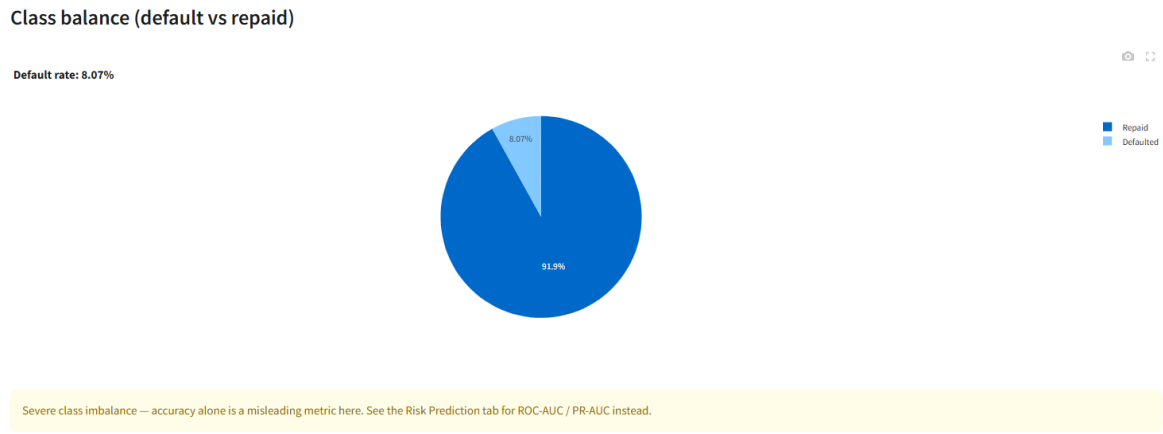
One of the first things I noticed digging into this dataset is that only about 8% of applicants actually default. That's a problem, because a model could technically be 92% "accurate" just by predicting everyone pays back their loan and never actually catch a real risky applicant. That kind of model would be useless for a bank.

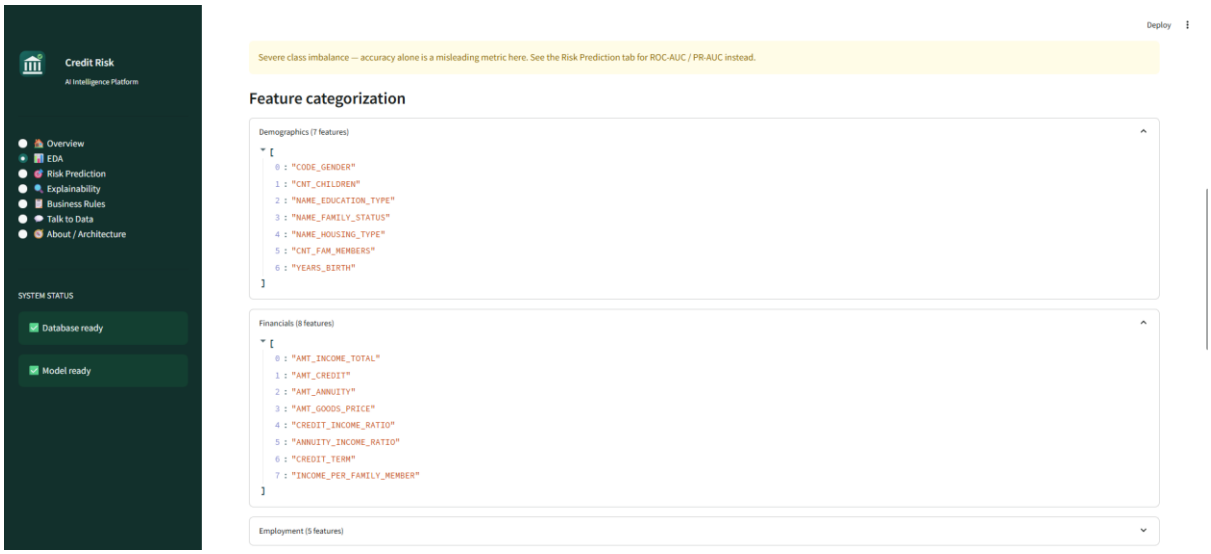
So instead of tweaking the dataset itself like copying existing defaulters over and over to artificially balance things out I told the model to just care more about getting defaulters right. Practically, that means every time it misses an actual default during training, it gets penalized about 11-12 times harder than when it misses a non-default. That pushes it to actually learn what a risky applicant looks like, instead of just taking the easy way out.

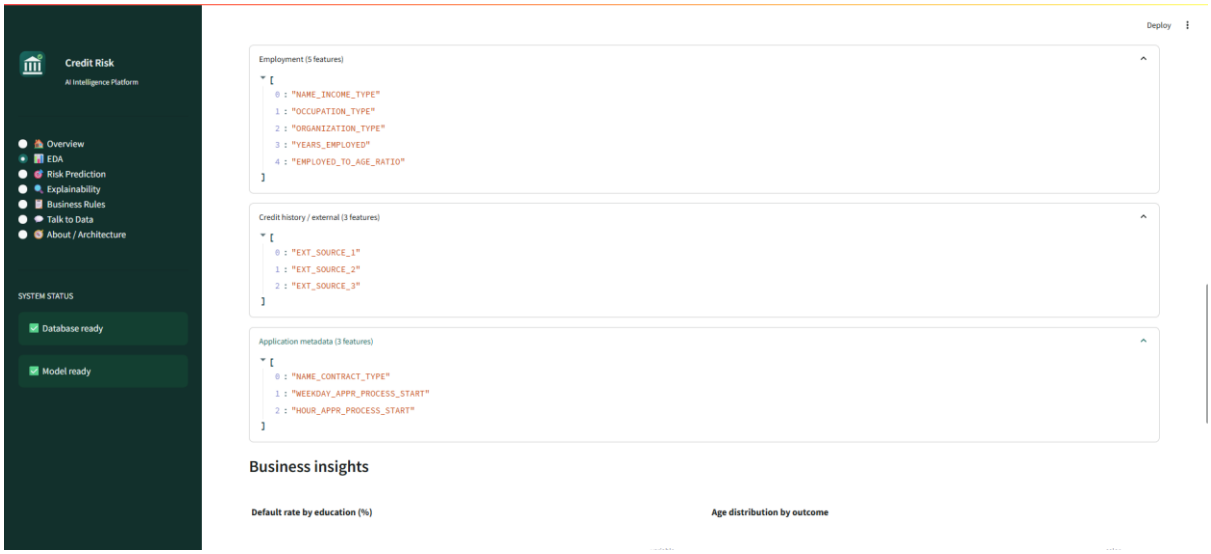
I went this route instead of oversampling (SMOTE) mainly because I didn't want to introduce fake, made-up applicants into the training data every row the model sees is a real person. It also makes the whole thing easier to explain: if someone in risk or

compliance asks 'how did you handle the imbalance,' the answer is one clean sentence, not 'we generated synthetic samples.'

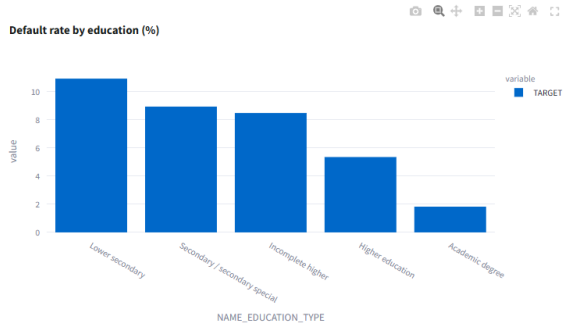
And since accuracy would just lie to me here, I mainly judged the model using PR-AUC, which is built specifically for cases where the thing you actually care about is rare like defaults.

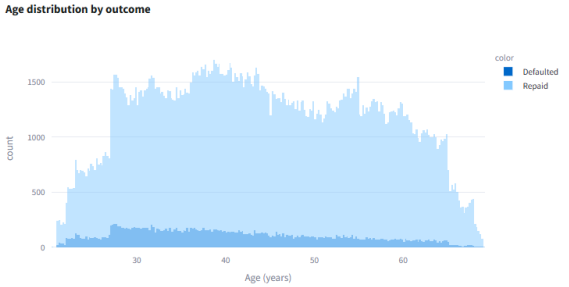






Business insights

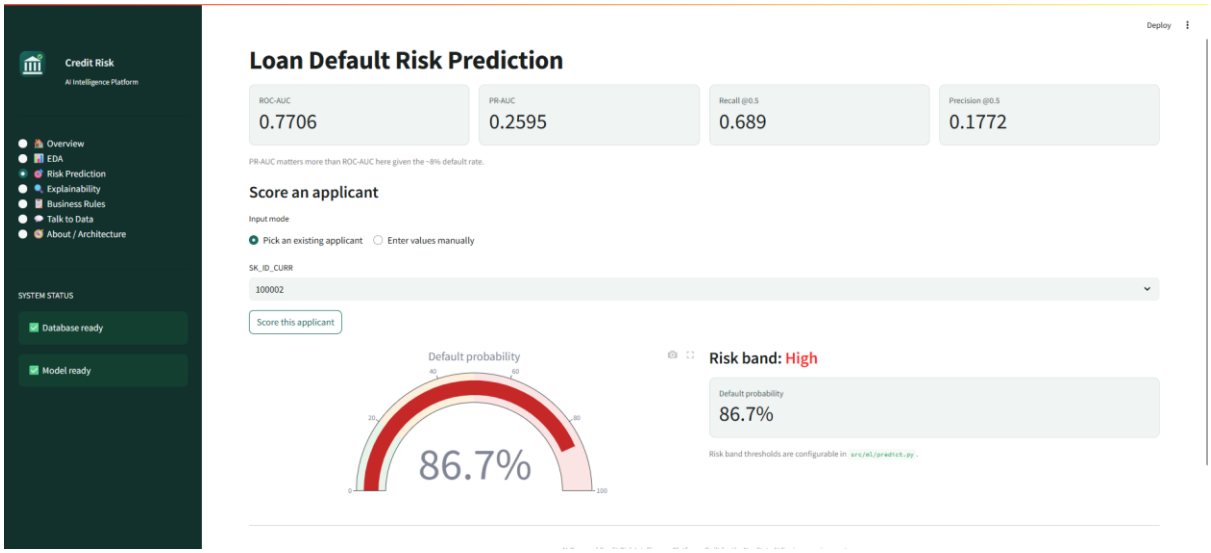


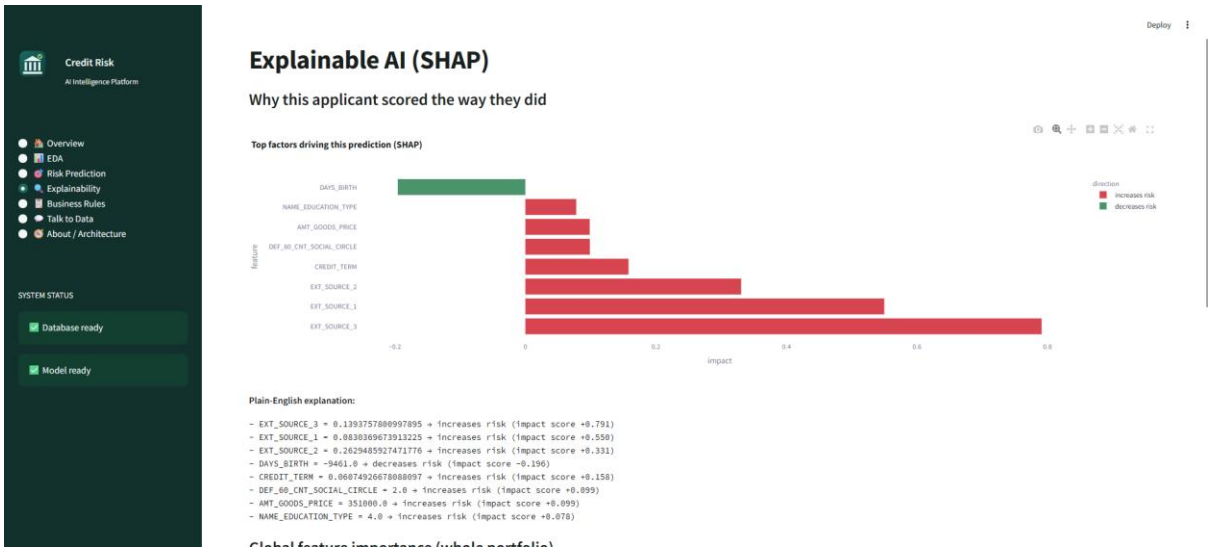


Correlation of external bureau scores with default:

	correlation
EXT_SOURCE_1	-0.1553
EXT_SOURCE_2	-0.1605
EXT_SOURCE_3	-0.1789

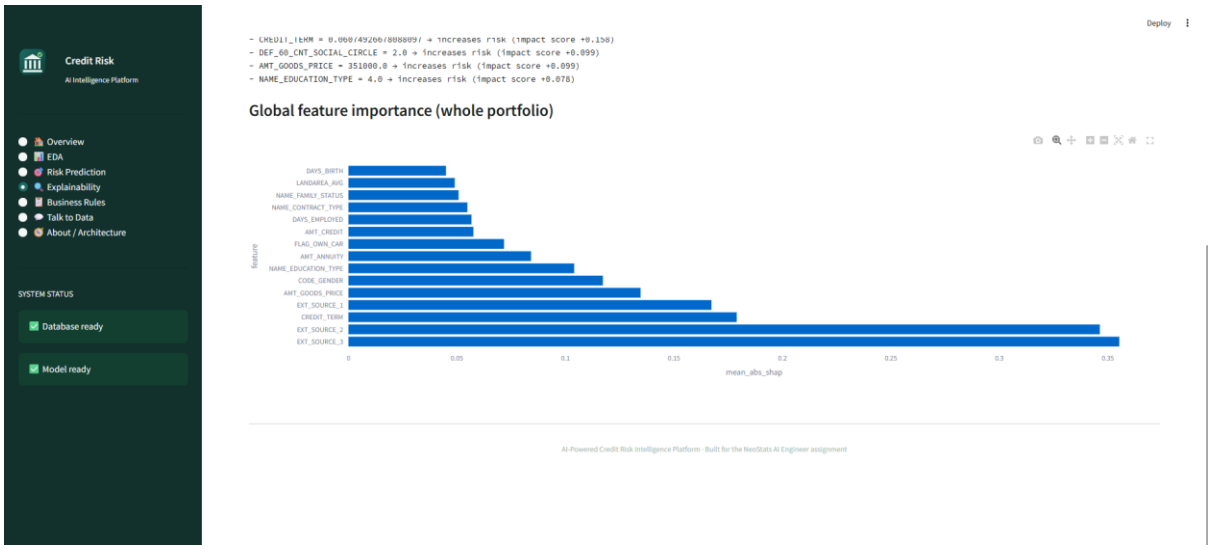
External bureau scores are consistently the strongest single predictors.

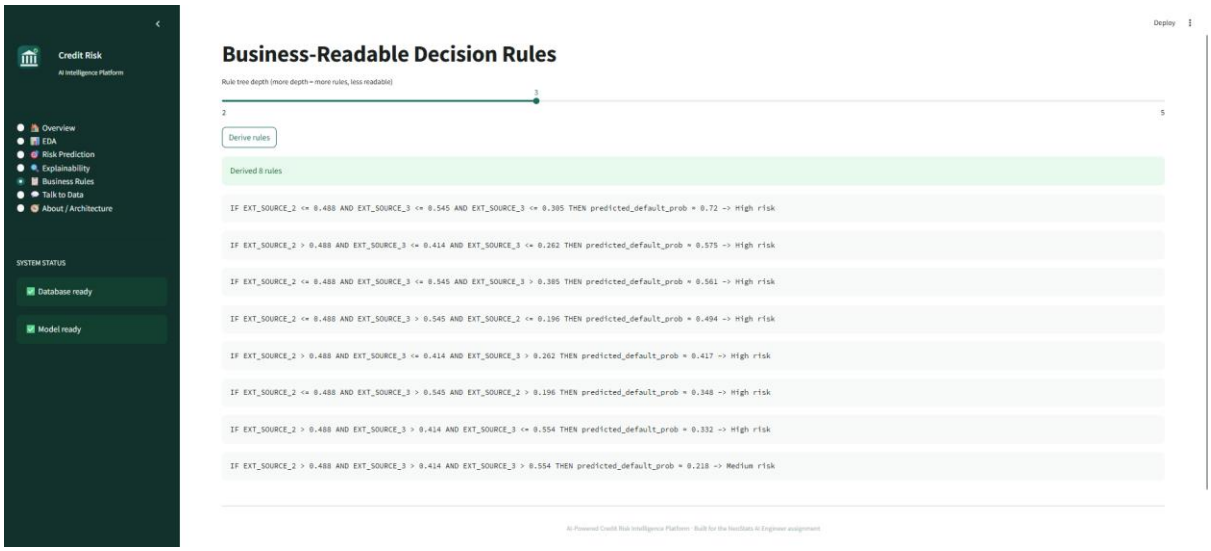


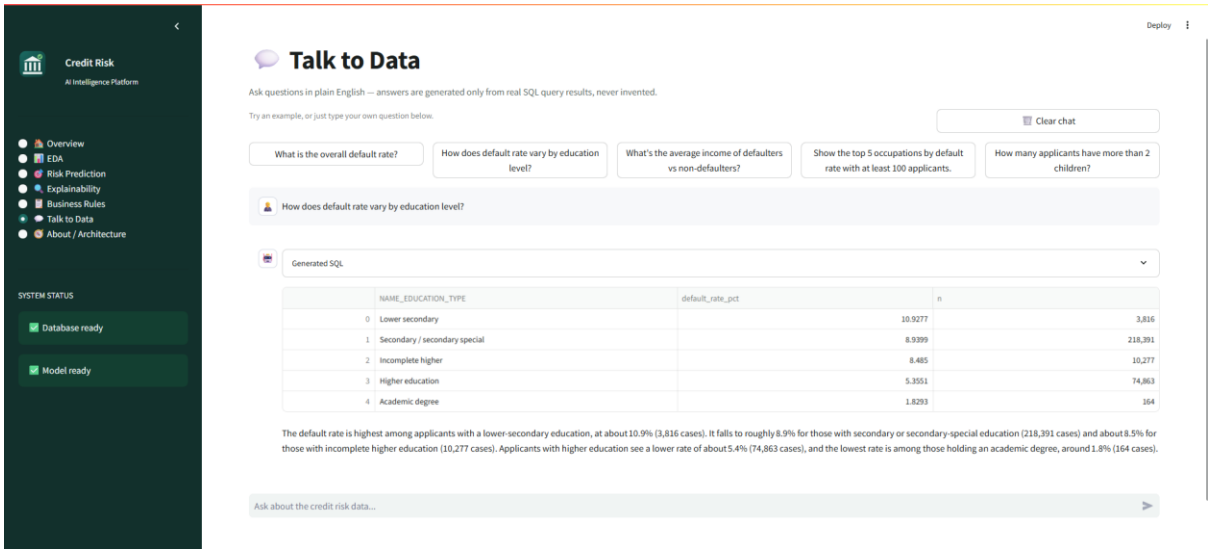


I picked SHAP mainly because the model itself is a black box it gives a score, not a reason. For a credit decision, 'the model said so' just isn't good enough; a bank needs to be able to explain to a customer or a regulator why someone was flagged

as high risk. SHAP breaks each prediction down into the actual factors that pushed the score up or down for that specific person, and because LightGBM is a tree-based model, I could use SHAP's exact tree explainer instead of an approximation so the explanations are precise, not just estimated.







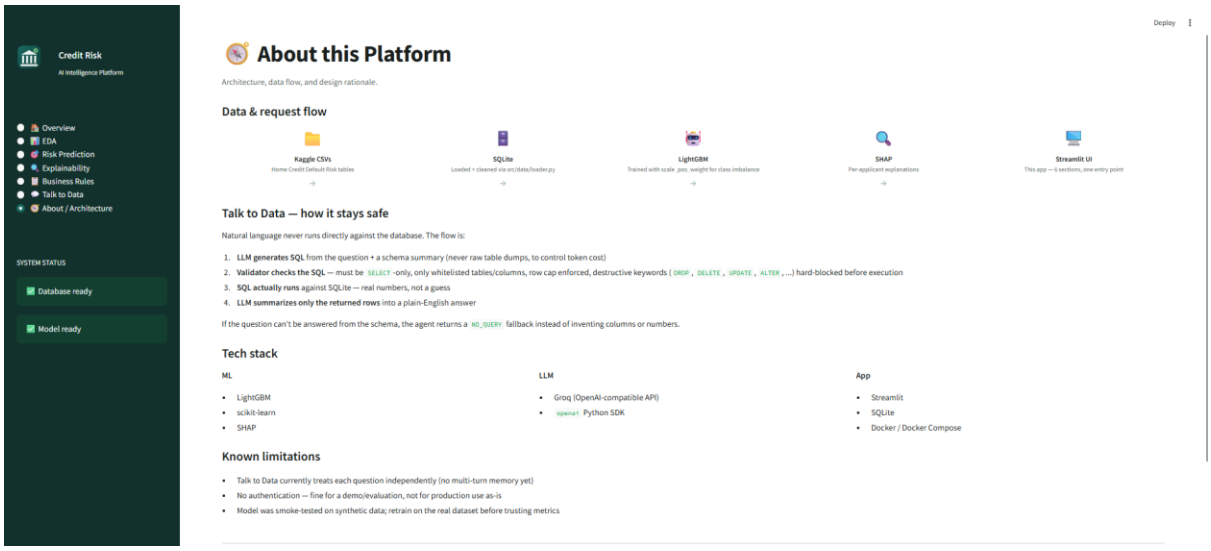
How does default rate vary by education level?

Generated SQL

```
SELECT NAME_EDUCATION_TYPE, AVG(TARGET)*100 AS default_rate_pct, COUNT(*) AS n FROM application GROUP BY NAME_EDUCATION_TYPE ORDER BY default_rate_pct DESC LIMIT 200
```

	NAME_EDUCATION_TYPE	default_rate_pct	n
0	Lower secondary	10.9277	3,816
1	Secondary / secondary special	8.9399	218,391
2	Incomplete higher	8.485	10,277
3	Higher education	5.3551	74,863
4	Academic degree	1.8293	164

The default rate is highest among applicants with a lower-secondary education, at about 10.9% (3,816 cases). It falls to roughly 8.9% for those with secondary or secondary-special education (218,391 cases) and about 8.5% for those with incomplete higher education (10,277 cases). Applicants with higher education see a lower rate of about 5.4% (74,863 cases), and the lowest rate is among those holding an academic degree, around 1.8% (164 cases).



Why LightGBM

Mostly because of the nature of the dataset, I decided to go with LightGBM for prediction. It's tabular, with many missing values and a combination of numerical and categorical features. It turns out gradient-boosted trees are excellent fits for that type

of data, and the LightGBM variant is also able to train quickly even on a lot of rows, handles missing values as a part of the algorithm without needing much preprocessing and offers feature importances out of the box. Besides that, it's a type of model that banks and credit-risk departments are already familiar with and use - no need for me to defend a different choice from the starting point. It's a tree model as well meaning it can fully support SHAP's exact explanation model, which was one of the main considerations in the requirement for transparency.

Why Groq

For the Talk-to-Data feature, I had started with Claude as the basis for my work but since no specific LLM had been assigned, I focused rather on the cost and convenience aspects. Groq emerged as the most feasible decision, it's completely free, doesn't require a credit card, and has no trial period limiting access like some other 'free' options that I found out were just free trial credits. Plus, Groq is a very flexible platform since it's OpenAI-API-compatible, swapping providers was nothing but updating the base-URL and the model-name, no code rewrite was necessary. Besides that, Groq's performance is pretty good with inference happening quite quick which for a chat interface is a plus since you wouldn't want your user to wait like 10 seconds for a SQL query to come back.

Why this class imbalance strategy

About 8% of the data are from defaulter cases, so accuracy can't be just considered on its own, a model which always predicts as 'no default' would appear as 92% accurate yet completely useless. I avoided using of oversampling methods like SMOTE, that create fake data for credit applicants, and I instead used LightGBM's scale_pos_weight option to train the learner so that defaulting a customer's loan is treated as being approximately 11-12 times more expensive. The reason why I opted for this approach is that it keeps the original data intact, model training happens only by the use of real cases, plus it allows me to use a very clear single number to tell them that rather than explain the complex technique of resampling. In addition, the way of evaluating that I was presenting reflected exactly the reality by starting with the metric PR-AUC instead of accuracy, because PR-AUC is better at detecting the capability of a model to catch the tiny positive class.